◆ ASTRAMIND

PRODUCT MANAGEMENT DOCUMENT

ASTraMind

AI-Powered Codebase Intelligence Platform

AUTHOR	Ishaan Verma
ROLE	Founder & Lead Engineer
VERSION	1.0 | May 2026
LIVE	astramind.vercel.app
REPO	github.com/ishaanv18/ASTraMind

TABLE OF CONTENTS

1 PRODUCT VISION & MISSION

VISION

"Make every developer feel like they have a senior engineer sitting beside them who has read every line of their codebase."

Mission

ASTraMind is an AI-powered codebase intelligence platform that transforms raw source code into a queryable, debuggable, and auditable knowledge graph — enabling developers to understand, maintain, and improve codebases **10x faster**.

Product Statement

*For software developers and engineering teams who spend too much time orienting themselves in unfamiliar codebases, ASTraMind is an AI intelligence layer that provides instant semantic understanding of any repository — unlike traditional IDE search tools that only match keywords, ASTraMind understands **meaning, context, and intent**.*

2 PROBLEM STATEMENT

The Core Problem

Developers spend **58% of their time reading and understanding code**, not writing it (GitHub Octoverse 2023). This is compounded by:

- **Onboarding friction:** A new engineer joining a team takes 3–6 months to become fully productive
- **Context switching cost:** Returning to a module after weeks away requires re-reading hundreds of files
- **Invisible architecture:** Business logic, security vulnerabilities, and architectural debt are buried in code
- **Tool fragmentation:** Teams use 5–7 separate tools for debugging, security, review, documentation

Problem Severity Matrix

Pain Point	Frequency	Impact
Cannot find where a feature is implemented	Daily	High
Time wasted onboarding to new codebase	Per hire	Critical
Security vulnerabilities caught late	Per sprint	Critical
Architecture decisions undocumented	Always	High
Manual code review creates bottlenecks	Every PR	Medium

3 TARGET USERS & PERSONAS

■ Priya, 23 — The Overwhelmed New Joiner	PRIMARY
Context: Just joined a 50k-line Python microservices company. No documentation. Senior devs too busy.	
Goal: Understand codebase quickly, contribute within first week, avoid breaking production.	
Pain Points: Endless grepping, constant Slack interruptions, feeling lost for weeks.	
ASTraMind Value: Semantic search lets Priya ask *"where is payment processing handled?"* and get exact file, line, and function name in seconds.	

■ Arjun, 26 — The Solo Indie Hacker	PRIMARY
Context: Freelance developer managing 5 client codebases simultaneously.	
Goal: Remember what he built 3 months ago, catch bugs before clients do, ship fast.	
Pain Points: No time to write docs, expensive consultants for code review, constant context-switching.	
ASTraMind Value: Pair Programmer + Debug features provide an always-available expert reviewer at zero cost.	

■ Kavya, 31 — The Overwhelmed Tech Lead	SECONDARY
Context: Engineering Manager leading a team of 8, security audit coming next month.	
Goal: Maintain code quality, unblock team, pass security audit without expensive tooling.	
Pain Points: Code review is a bottleneck. Security scanning tools cost $200+/month.	
ASTraMind Value: Automated Code Review + Security Sentinel reduce her review load by 60%.	

■ Marcus, 29 — The Open Source Explorer	TERTIARY
Context: Senior Developer who contributes to multiple open source projects.	
Goal: Understand a new repo within 30 minutes before contributing a PR.	
Pain Points: No consistent tooling for quickly exploring unfamiliar large repositories.	
ASTraMind Value: Index any public GitHub URL. Onboarding Copilot explains architecture immediately.	

4 COMPETITIVE ANALYSIS

Feature	ASTraMind	GitHub Copilot	Cursor	Sourcegraph	Tabnine
Semantic code search	✓ YES	✗ NO	Partial	✓ YES	✗ NO
Full repo indexing	✓ YES	✗ NO	✓ YES	✓ YES	✗ NO
Security scanning	✓ YES	✗ NO	✗ NO	Partial	✗ NO
Architecture analysis	✓ YES	✗ NO	✗ NO	✗ NO	✗ NO
ADR generation	✓ YES	✗ NO	✗ NO	✗ NO	✗ NO
Code Time Machine	✓ YES	✗ NO	✗ NO	✗ NO	✗ NO
Free tier (real)	✓ YES	NO ($10/mo)	NO ($20/mo)	Partial	Partial
Works on any public repo	✓ YES	IDE only	IDE only	✓ YES	IDE only
Web-based / no install	✓ YES	✗ NO	✗ NO	✓ YES	✗ NO

Key Differentiators

- **Breadth:** 12+ intelligence features in one platform — competitors specialize in 1–2
- **Accessibility:** Web-based, no IDE plugin required, any public GitHub repo
- **Depth:** AST-level parsing gives semantic understanding, not just text search
- **Free:** Full feature access with no credit card required

5 PRODUCT REQUIREMENTS DOCUMENT (PRD)

FR-01 Repository Indexing — P0

Accepts a GitHub URL or local path, clones the repository, parses all source files using AST analysis, generates vector embeddings, and stores them in a vector database.

- Supports Python, JavaScript, TypeScript, Java, Go, Rust, C++
- Files >500 KB skipped with a logged warning
- Indexing progress reported in real-time (0–100%)
- Completes within 3 minutes for repos up to 200 files
- Memory usage during indexing ≤ 400 MB

FR-02 Semantic Search — P0

Users query the indexed codebase in natural language and receive ranked, contextually relevant code snippets with file paths and line numbers.

- Search returns results in <2 seconds
- Results include file path, line range, function name, and code snippet
- Results ranked by cosine similarity score
- Minimum 5 results returned when available

FR-03 AI-Powered Debugging — P0

User describes a bug in natural language; system retrieves relevant code context and returns a structured diagnosis with root cause, affected files, and fix suggestions.

- Response includes probable root cause, relevant code files, suggested fix
- Response generated within 10 seconds
- Response references actual function and file names from the codebase

FR-04 Security Scanning — P1

Analyzes indexed codebase for OWASP Top 10 vulnerability patterns and returns a prioritized list of findings with severity ratings.

- Detects: SQL injection, hardcoded secrets, XSS, insecure dependencies, missing auth checks
- Findings categorized as Critical / High / Medium / Low
- Each finding includes description, affected file, line number, remediation advice

FR-05 Code Review Assistant — P1

User submits a code diff or file; system returns a structured review with quality score, issues, and improvement suggestions covering logic, performance, readability, and security.

FR-06 Architecture Guardian — P1

Generates a high-level architecture overview including component map, entry points, and dependency analysis.

FR-07 Onboarding Copilot — P1

Generates a structured onboarding guide for new developers — covering architecture, key files, setup steps, core workflows, and common gotchas.

FR-08 GitHub Integration — P0

User connects GitHub via Personal Access Token; system stores it in HTTP-only cookie and allows browsing and importing of all repositories.

- Token stored in HTTP-only Secure SameSite=None cookie (XSS-safe)
- Token never exposed to frontend JavaScript
- All user repos listed within 5 seconds of auth

FR-09 Workspace Manager — P0

Dashboard showing all indexed repositories with ability to select, switch, and delete workspaces.

- Lists repos filtered by logged-in GitHub user
- Delete removes data from Postgres, Qdrant, and cache
- Repo selection persists across page refreshes

Non-Functional Requirements

Requirement	Target
API response time (p95)	Under 3 seconds for AI endpoints
Peak memory during indexing	Under 400 MB
API availability	99%+ uptime
Search latency	Under 2 seconds
CORS policy	Requests from astramind.vercel.app only

6 USER STORIES

ID	Role	Goal	Given / When / Then
US-001	Developer	Connect GitHub account	**Given** I am on the GitHub tab **When** I enter my PAT and click Connect **Then** I see all my repositories within 5 seconds
US-002	Developer	Import a public GitHub repo by URL	**Given** I am on the Local/URL tab **When** I enter a GitHub URL and click Start Indexing **Then** Indexing starts with real-time progress shown
US-003	Developer	Delete an indexed repository	**Given** A repo is indexed **When** I click Delete **Then** The repo is removed; vectors deleted from Qdrant; records deleted from Postgres
US-004	Developer	Search codebase in plain English	**Given** A repository is selected **When** I type "where is user authentication handled?" **Then** I see ranked results with file paths, function names, and code snippets
US-005	Developer	Describe a bug and get a diagnosis	**Given** A repo is indexed **When** I describe a bug **Then** I receive a structured analysis referencing actual session management code
US-006	Team Lead	Run a security scan	**Given** A repo is indexed **When** I click Run Security Scan **Then** I receive a prioritized list of vulnerabilities with severity and remediation steps
US-007	New Engineer	Generate onboarding guide	**Given** A repo is indexed **When** I click Generate Onboarding Guide **Then** I receive a structured guide covering architecture, key files, and common patterns

7 FEATURE PRIORITIZATION — MOSCOW

Must Have — MVP	
✓ Repository indexing via GitHub URL	
✓ Semantic code search	
✓ AI-powered debugging	
✓ GitHub PAT authentication with HTTP-only cookies	
✓ Workspace Manager (list, select, delete repos)	
✓ Real-time indexing progress	

Should Have — V1.0	
✓ Security Sentinel (OWASP vulnerability scanning)	
✓ Code Review Assistant	
✓ Architecture Guardian	
✓ Onboarding Copilot	
✓ Natural Language Query	
✓ Quality Trends dashboard	

Could Have — V1.5	
✓ Code Time Machine (commit history analysis)	
✓ ADR Generator	
✓ Pair Programmer (multi-turn chat)	
✓ Commit Intelligence	

Will Not Have This Cycle	
— GitHub OAuth 2.0 (using PAT for now)	
— VS Code extension	
— Team collaboration (shared workspaces)	
— Self-hosted deployment	
— Billing and subscription management	

8 PRODUCT ROADMAP

Phase 1: Foundation	Days 1–4
✓ Project scaffold FastAPI + React	Done
✓ Tree-sitter AST parser for 7 languages	Done
✓ ChromaDB local vector store	Done
✓ Basic semantic search endpoint	Done
✓ React frontend with Workspace Manager	Done

Phase 2: Intelligence Layer	Days 5–8
✓ Groq LLM integration	Done
✓ Security Sentinel + OWASP prompts	Done
✓ Code Review, Architecture Guardian, Onboarding Copilot	Done
✓ ADR Generator, Code Time Machine, Pair Programmer	Done
✓ Commit Intelligence, Quality Trends, NL Query	Done

Phase 3: Cloud Production	Days 9–11
✓ Supabase PostgreSQL schema and async client	Done
✓ Qdrant Cloud integration	Done
✓ GitHub PAT auth with HTTP-only cookies	Done
✓ Dockerfile and render.yaml configuration	Done
✓ Vercel deployment with SPA routing + CORS configuration	Done

Phase 4: Stability & Performance	Days 12–14
✓ Fix PgBouncer DuplicatePreparedStatementError	Done
✓ Replace PyTorch with FastEmbed (1.5 GB → 150 MB RAM)	Done
✓ 4-phase batch indexing pipeline	Done
✓ OOM fix with BATCH_SIZE=20 and gc.collect()	Done
✓ Migrate Supabase to Neon (eliminates free-tier pausing)	Done
✓ Fix DB delete cascade + GitHub repo list refresh bug	Done

9 SUCCESS METRICS & KPIS

500+	50	100	>60%
Unique Visitors / Month	GitHub Stars Target	Repo Imports Target	Activation Rate

Acquisition

Metric	Target
Unique visitors monthly	**500**
GitHub stars	**50**
Total repository imports	**100**

Activation

Metric	Target
Activation rate	**>60%**
Time to first search	**<5 minutes**
Indexing success rate	**>95%**

Engagement

Metric	Target
Features used per session	**3 or more**
Return visit rate (7-day)	**>40%**
Average session duration	**>8 minutes**

Technical Performance

Metric	Target
Indexing time for 131-file repo	**<3 minutes**
Search response time p95	**<2 seconds**
AI feature response p95	**<10 seconds**
Peak memory during indexing	**<400 MB**
Backend uptime	**>99%**

10 SPRINT PLANNING

Sprint 1 — Core Foundation (Days 1–3)	Points	Status
FastAPI project structure and routing	2	✓ Done
Tree-sitter AST parser (7 languages)	5	✓ Done
Chunk and embed pipeline with ChromaDB	5	✓ Done
Semantic search endpoint	3	✓ Done
React frontend scaffold with repo manager	3	✓ Done
TOTAL	18	

Sprint 2 — AI Intelligence Layer (Days 4–6)	Points	Status
Groq LLM integration and prompt system	3	✓ Done
Debug endpoint with RAG context	3	✓ Done
Security Sentinel OWASP prompts	5	✓ Done
Code Review endpoint	3	✓ Done
Architecture Guardian	4	✓ Done
Onboarding Copilot	3	✓ Done
ADR Generator	3	✓ Done
Pair Programmer multi-turn chat	4	✓ Done
Code Time Machine	4	✓ Done
TOTAL	32	

Sprint 3 — Cloud & Authentication (Days 7–9)	Points	Status
Supabase PostgreSQL schema + async client	5	✓ Done
Qdrant Cloud integration	5	✓ Done
GitHub PAT auth with HTTP-only cookie	4	✓ Done
Dockerfile and render.yaml	3	✓ Done
Vercel deployment and SPA routing	2	✓ Done
CORS configuration	2	✓ Done
TOTAL	21	

Sprint 4 — Bug Fixes & Optimization (Days 10–14)	Points	Status

Fix DuplicatePreparedStatementError (PgBouncer)	8	✓ Done
Replace PyTorch with FastEmbed	5	✓ Done
4-phase batch indexing pipeline	5	✓ Done
OOM fix with micro-batching + gc.collect()	5	✓ Done
Postgres DB delete cascade fix	3	✓ Done
Migrate Supabase to Neon	3	✓ Done
Frontend GitHub reload fix	2	✓ Done
TOTAL	**31**	

Total Story Points Delivered: **102** across 4 sprints in 14 days.

11 RISK REGISTER

Risk	Likelihood	Impact	Mitigation
Free-tier DB pauses (Supabase)	High	Critical	Migrated to Neon which never pauses
Free-tier OOM on Render 512 MB	High	Critical	FastEmbed + micro-batch + gc.collect()
Qdrant cluster deleted on inactivity	Medium	High	Documented recreation steps; 2-minute fix
GitHub PAT token theft via XSS	Low	Critical	HTTP-only Secure SameSite=None cookie
Groq API rate limits	Medium	High	slowapi rate limiting; graceful errors
Render 15-minute cold start	High	Medium	Shown as warning in UI; acceptable for demo
Large repo timeout during indexing	Medium	High	MAX_FILE_SIZE_KB=500; binary file skip
LLM hallucination on code context	Medium	Medium	RAG grounds all responses in real code

12 ARCHITECTURAL DECISION RECORDS (ADR LOG)

ADR-001	FastEmbed instead of PyTorch / sentence-transformers	Accepted · Sprint 4
Context	Backend OOM on Render 512 MB RAM; PyTorch stack required 1.5 GB.	
Decision	Replace with ONNX-based FastEmbed requiring only 150 MB.	
Result	90% memory reduction; maintained acceptable embedding quality.	

ADR-002	Migrate from Supabase to Neon Postgres	Accepted · Sprint 4
Context	Supabase PgBouncer transaction mode incompatible with asyncpg prepared statements; free tier pauses after 1 week.	
Decision	Neon Postgres with direct connection, standard QueuePool, ssl=True.	
Result	Eliminated DuplicatePreparedStatementError; eliminated free-tier pausing.	

ADR-003	Micro-batched embedding (BATCH_SIZE=20 with gc.collect)	Accepted · Sprint 4
Context	Batch of 1000+ chunks spiked RAM past 512 MB causing silent OOM, presenting as CORS 502.	
Decision	Process 20 chunks at a time, explicitly free arrays with del + gc.collect() between batches.	
Result	Peak memory flat at 200 MB; stable indexing for 131+ file repos.	

ADR-004	HTTP-only cookie for GitHub PAT	Accepted · Sprint 3
Context	Storing PAT in localStorage or Zustand exposes it to XSS.	
Decision	Backend validates PAT once, stores in HTTP-only Secure SameSite=None cookie.	
Result	Token inaccessible to JavaScript; requires SameSite=None for Vercel-to-Render cross-origin.	

ADR-005	Tree-sitter for AST parsing	Accepted · Sprint 1
Context	Naive character-based text splitting destroys code logic when a function is split mid-body.	
Decision	Use tree-sitter with language-specific grammars to identify function boundaries semantically.	
Result	Semantically correct chunks; supports 7 languages; significantly better search results.	

13 GO-TO-MARKET STRATEGY

Target Channels

- **Reddit:** r/programming, r/webdev — organic community posts
- **Hacker News:** Show HN launch with technical deep-dive
- **GitHub:** Strong README, demo GIF, relevant topic tags for Explore discovery
- **Twitter/X:** Short demo video showcasing 3D UI and instant semantic search
- **Product Hunt:** Launch with compelling tagline and screenshots
- **College Networks:** Demo at hackathons, CS societies, placement preparation groups

Positioning

> ***"The AI that reads your entire codebase in minutes — so you do not have to."***

Pricing Strategy (Future)

Tier	Price	Limits
Free	$0	3 repos, 5 AI queries per day
Pro	$12 / month	Unlimited repos, unlimited queries
Team	$49 / month	5 seats, shared workspaces, priority support

14 USER JOURNEY MAP

AWARENESS	User sees GitHub repo or Product Hunt post
INTEREST	Visits astramind.vercel.app — impressed by 3D DataCore animation and UI
CONSIDERATION	Reads feature list — security scanning, architecture analysis, all free
ACTIVATION	Connects GitHub, imports a real project, sees first semantic search result pointing to exact file and line
ENGAGEMENT	Uses Debug, Security, Code Review on real work tasks; saves measurable time
RETENTION	Returns next day for another repository; shares with a colleague
ADVOCACY	Posts on Twitter, stars the GitHub repo, recommends to team

15 RETROSPECTIVE — LESSONS LEARNED

What Went Well

- Technology choices paid off: FastEmbed, Tree-sitter, and Qdrant were the right tools
- FastAPI async architecture allowed background indexing without blocking the API
- 3D DataCore UI with glassmorphism created a strong first impression
- Rapid iteration: 4 sprints from zero to production in 14 days

What Was Hard

- Infrastructure debugging: CORS errors masking OOM crashes required systematic root cause analysis
- PgBouncer incompatibility: asyncpg prepared statement conflict required deep research into SQLAlchemy internals
- Memory constraints: Fitting a serious AI pipeline into 512 MB required creative, non-obvious solutions

What I Would Do Differently

- Start with Neon instead of Supabase from day one (would have saved 8+ hours of debugging)
- Implement memory profiling on day one before choosing embedding strategy
- Plan for free-tier limitations explicitly in initial architecture review
- Add Redis cache from the beginning instead of relying on in-memory fallback

Key Engineering Insights

1	CORS errors from the browser are often a server crash in disguise — always check the raw HTTP status code first.
2	PgBouncer transaction mode and asyncpg prepared statements are fundamentally incompatible — use session mode or switch providers.
3	On memory-constrained servers, explicit garbage collection between batch operations is not premature optimization — it is necessary engineering.
4	Batch size for embedding should be tuned to peak memory spike, not average chunk size.